Memoria Técnica: Control Avanzado de Turtlesim

Práctica 1 — Asignatura de Robótica (ROS 2 Jazzy Jalisco)

Alumno:	**Javier Ferreño Corzo**	**Titulación:**	Grado en Ingeniería de Computadores
Centro:	E.T.S.I. de Sistemas Informáticos (UPM)	**Paquete:**	`turtle_control` (Nodo: `teleop_turtle`)

1. Introducción y Objetivos

El objetivo de esta práctica es desarrollar un paquete en **ROS 2** que extienda el simulador `turtlesim`. Frente al nodo por defecto `turtle_teleop_key` (que bloquea la consola y sólo admite pasos ortogonales discretos), esta solución proporciona una cinemática diferencial continua capaz de **avanzar y rotar de forma simultánea** (trazado fluido de curvas y círculos), interactuar con los **servicios** nativos del simulador mediante teclado y posibilitar la **reconfiguración en caliente de parámetros** en tiempo de ejecución.

2. Funciones Implementadas para el Control del Robot

La solución se encapsula en el nodo `TeleopTurtle` (desarrollado en `turtle_control/teleop_turtle.py`) estructurado en los siguientes componentes:

2.1. Captura No Bloqueante y Movimiento Simultáneo (W, S, A, D)

Se integra `pynput.keyboard.Listener` en un hilo desacoplado (*daemon thread*), evitando bloquear el bucle de eventos principal de ROS 2.

- **Conjunto de teclas concurrentes:** El nodo almacena las pulsaciones activas en `self.pressed_keys = set()`. En `on_key_press` se añade la tecla (`w`, `s`, `a`, `d`) y en `on_key_release` se retira de inmediato.
- **Cálculo cinemático diferencial a 30 Hz (`timer_callback`):** Un temporizador periódico publica continuamente en `/turtle1/cmd_vel` el mensaje `geometry_msgs/msg/Twist` sumando los componentes activos:

```
linear = (self.speed if 'w' in self.pressed_keys else 0.0) - (self.speed if 's' in self.pressed_keys else 0.0)
angular = (self.angular_speed if 'a' in self.pressed_keys else 0.0) - (self.angular_speed if 'd' in self.pressed_k
twist.linear.x, twist.angular.z = float(linear), float(angular)
self.cmd_vel_pub.publish(twist)
```

  Al presionar conjuntamente `W` y `A` (o `W` y `D`), el simulador recibe tanto aceleración lineal como giro angular simultáneos, generando curvas naturales continuas.

2.2. Invocación Asíncrona de Servicios de Turtlesim

Las peticiones se realizan de forma no bloqueante con `call_async()` para no pausar el temporizador de velocidad:

- **Control de Pluma (ESPACIO → `/turtle1/set_pen`):** La función `toggle_pen()` conmuta `self.pen_enabled` y envía `SetPen.Request` fijando `req.off = 0` (activar trazo) o `req.off = 1` (desactivar trazo).
- **Limpieza de Pantalla (C → `/clear`):** La función `call_clear()` envía una solicitud de tipo `std_srvs/srv/Empty`, borrando las líneas dibujadas sobre el lienzo sin alterar las coordenadas del robot.
- **Reinicio Central Cenital (R → `/turtle1/teleport_absolute`):** El comando nativo `/reset` orienta la tortuga a 0 rad (derecha). Para cumplir la especificación (*"al centro y mirando hacia arriba"*), la función `call_reset_center_up()` utiliza `TeleportAbsolute.Request` con `x=5.5444`, `y=5.5444` y `theta = π / 2` (90° exactos).

2.3. Parámetros Dinámicos en Tiempo de Ejecución

El nodo declara formalmente: `speed` (def. 2.0), `angular_speed` (def. 2.0) y `log_level` (def. 'INFO'). Mediante `self.add_on_set_parameters_callback(self.on_parameter_change)`, el nodo captura en caliente cualquier modificación realizada con `ros2 param set`, actualizando la velocidad o el nivel del logger (`self.get_logger().set_level(...)`) al instante.

2.4. Automatización con Lanzador (Launch File)

El archivo `launch/turtlesim_teleop.launch.py` declara los argumentos configurables (`DeclareLaunchArgument`) e inicializa conjuntamente `turtlesim_node` y `teleop_turtle` con `emulate_tty=True` para habilitar la interacción directa desde consola.

3. Interfaces y Comunicaciones en el Grafo ROS 2

Canal / Mecanismo	Nombre	Tipo de Interfaz	Función y Propósito
Topic (Publisher)	`/turtle1/cmd_vel`	`geometry_msgs/msg/Twist`	Emisión continua (30 Hz) de velocidades lineal X y angular Z.
Service Client	`/turtle1/set_pen`	`turtlesim/srv/SetPen`	Activa o desactiva el trazo de pintura (tecla `ESPACIO`).
Service Client	`/clear`	`std_srvs/srv/Empty`	Borra todas las trayectorias dibujadas (tecla `C`).
Service Client	`/turtle1/teleport_absolute`	`turtlesim/srv/TeleportAbsolute`	Posiciona en centro `(5.54, 5.54)` mirando al norte `θ = π/2` (tecla `R`).
Parámetros	`speed` / `angular_speed`	`Double`	Control en caliente de la velocidad de traslación y giro.
Parámetro	`log_level`	`String`	Ajuste dinámico del nivel de log (DEBUG, INFO, WARN, ERROR).

4. Guía de Compilación y Comandos de Ejecución

1. Compilación del espacio de trabajo (compilación limpia y verificada):

```
cd ~/ROS/practica\ 1    # o dentro del directorio del workspace ws
colcon build --packages-select turtle_control
source install/setup.bash
```

2. Ejecución mediante el lanzador integrado:

```
# Lanzamiento unificado con parámetros por defecto:
ros2 launch turtle_control turtlesim_teleop.launch.py

# Lanzamiento alternativo modificando velocidad inicial o severidad de log:
ros2 launch turtle_control turtlesim_teleop.launch.py speed:=3.5 log_level:=DEBUG
```

3. Modificación interactiva de parámetros en caliente:

```
ros2 param set /teleop_turtle speed 5.0        # Incrementa la velocidad lineal en caliente
ros2 param set /teleop_turtle log_level DEBUG   # Modifica la verbosidad de salida del nodo
```

5. Matriz de Cumplimiento de Requisitos del Enunciado

Requisito del Enunciado	Estado	Mecanismo Técnico Implementado
Movimiento continuo con W, S, A, D	CUMPLIDO	Mapeo de teclas sobre vector `geometry_msgs/Twist`.
Giro y avance simultáneo (curvas continuas)	CUMPLIDO	Set concurrente con `pynput` y publicación periódica combinada a 30 Hz.
Tecla SPACE (habilitar/deshabilitar trazo)	CUMPLIDO	Cliente asíncrono para `/turtle1/set_pen` con flag conmutable `off: 0/1`.
Tecla C (borrado de dibujo)	CUMPLIDO	Cliente asíncrono para servicio nativo `/clear`.
Tecla R (centro y mirando hacia arriba)	CUMPLIDO	Teletransporte a `x=5.544, y=5.544, θ=π/2` con `/turtle1/teleport_absolute`.
Lanzador dedicado (Launch File)	CUMPLIDO	`turtlesim_teleop.launch.py` lanza conjuntamente simulador y control.
Parámetro de nivel de logging	CUMPLIDO	Parámetro reactivo `log_level` con callback dinámico de severidad.
Parámetro de velocidad en tiempo real	CUMPLIDO	Parámetro `speed` reactivo mediante callback ante `ros2 param set`.